TECHNISCHE UNIVERSITÄT
ILMENAU

Institut für Technische Informatik und Ingenieurinformatik
Fakultät für Informatik und Automatisierung
Fachgebiet System- und Software-Engineering

Bachelorarbeit

Modellgetriebene Microservice-Entwicklung

vorgelegt von

Patrik Löffler
Matrikelnummer 56207

Betreuer:

Prof. Armin Zimmermann
Dr. Ralph Maschotta, Tino Jungebloud

Ilmenau, den 2. November 2023

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Einführung in das Thema 1

1.2 Zielstellung 1

2 Grundlagen **3**

2.1 Domain-Driven Design 3

2.1.1 Definition 3

2.1.2 Bausteine des Model-Driven Design 4

2.1.3 Strategic Design 7

2.2 Microservices 10

2.2.1 Geschichte und Definition 10

2.2.2 Eigenschaften von Microservices 11

2.2.3 Architektur von Microservice-Systemen 13

2.2.4 Kommunikation 18

2.2.5 Betrieb und Continious Delivery 21

2.3 Cloud-Infrastruktur 23

2.3.1 Definition 23

2.3.2 Docker 23

2.3.3 Kubernetes 24

2.4 Modellgetriebene Entwicklung von Microservices 26

2.5 Noch zu sichten 27

2.5.1 ARGON - automated cloud infrastructure provisioning using a domain-specific language 27

2.5.2 model-driven approach for adaptive microservice-based systems using VxBPMN4MS 27

2.5.3 Context Mapper, a tool for DDD and microservice design. 28

2.5.4 DOML simplifies infrastructure deployments by unifying multiple IaC languages 29

1 Einleitung

1.1 Einführung in das Thema

1.2 Zielstellung

Ziel der Arbeit ist es, Anwendungen, die das Microservice-Paradigma verwenden, mit Werkzeugen und Methoden der modellgetriebenen Softwareentwicklung zu entwickeln.

Dazu wird ein Metamodell konzipiert, welches die grundlegenden Eigenschaften des Paradigmas abstrahiert. Ergänzt wird dies durch die Berücksichtigung von Konzepten des Domain-Driven Design zur Modellierung der Geschäftslogik.

Die mithilfe dieses Metamodells generierten Anwendungen sollen in der Lage sein, bereits existierende, sogenannte Microservice Chassis – wiederverwendbare Grundbausteine eines Microservices – zu verwenden. Daraufhin wird erforscht, welche Möglichkeiten es gibt, diese generierten Anwendungen automatisiert, in Abhängigkeit von Versionsaktualisierungen der Chassis, zu migrieren.

Außerdem wird untersucht, inwiefern die der Anwendung zugrunde liegenden IT-Infrastruktur in solch einem Metamodell berücksichtigt werden kann und welche Möglichkeiten sich daraus ergeben.

2 Grundlagen

2.1 Domain-Driven Design

2.1.1 Definition

„Domain-Driven Design“ (Abk. DDD) ist eine Herangehensweise, Problemdomänen in ein Modell für die Softwareentwicklung zu überführen.

Maßgeblich geprägt wurde das DDD durch Eric Evans in seinem Buch „Domain-Driven Design: Tackling Complexity in the Heart of Software“ [Eva07]. Kern des domänengetriebenen Entwurfs ist es, die reale, oft unübersichtliche Fachlichkeit fortschreitend in feiner geschnittene Teilbereiche zu kapseln. Diese sollten weiterhin möglichst wenige Abhängigkeiten zueinander haben [Dow18, S.63, 70]. Dieser auf das Domänenmodell fokussierte Ansatz versucht dabei, Kernaspekte einer Anwendung möglichst im Modell zu integrieren, anstatt sie über externe Dienste zu binden [Das17, S.74].

Evans stellt drei grundlegende Eigenschaften fest, welche bei der Wahl eines Modells im DDD zu berücksichtigen sind [Eva07, S. 3-4]:

- Es existiert immer eine Abhängigkeit zwischen Modell und Implementierung.
- Die Terminologie während der Softwareentwicklung ist eine Implikation des zugrunde liegenden Modells.
- Nützliches und aufgearbeitetes Wissen kann diesem Modell entnommen werden.

Ein weiteres Kernkonzept des DDD ist die „Ubiquitous Language“. Sie verbindet Entwickler und Domänen-Experten durch eine auf dem Modell basierenden Sprache. Dabei definiert sie eindeutig die verwendeten Begriffe und die ihnen zugeordneten Fachlichkeiten [Eva07, S. 25-27]. Ein weiterer Kern ist das „Model-Driven Design“ (Abk. MDD). MDD betont und bezeichnet das notwendige Ineinandergreifen der Konzeption eines Domänenmodells mit dem Entwurf der Anwendungssoftware. Um eine enge Übereinstimmung zwischen Modell und Software sicherzustellen, ist es von Vorteil, innerhalb eines durch Software-Tools unterstützten Modellierungsansatzes zu arbeiten. Objektorientierte Programmiersprachen eignen sich besonders gut, da sich die objekthaften Eigenschaften von Modellen gut abbilden lassen [Eva07, S. 49-51].

2.1.2 Bausteine des Model-Driven Design

Das DDD definiert verschiedene Bausteine des MDD [Eva07, S. 65]:

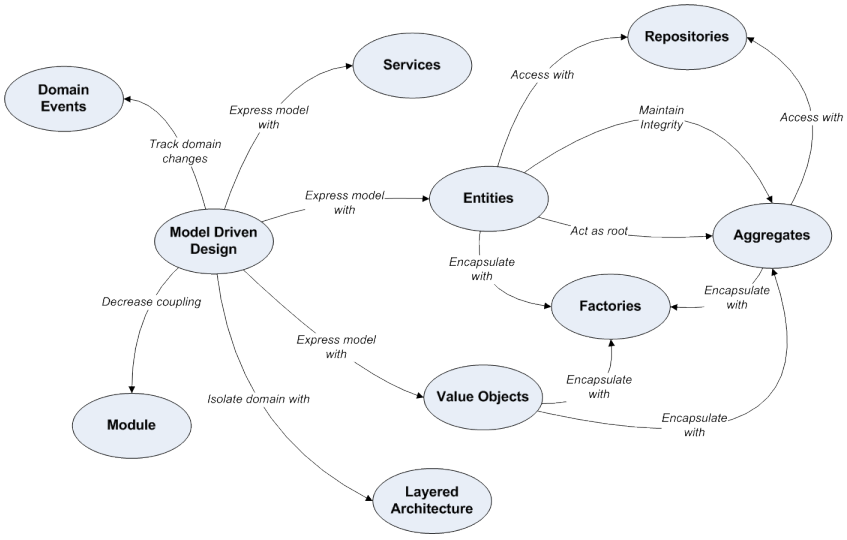

Abbildung 2.1: Bausteine des Model-Driven Design

Schichtenarchitektur

Schichtearchitekturn helfen die Komplexität von Software zu beherschen. Jedes Element einer Schicht hängt nur von Elementen derselben oder einer tieferen Schicht ab. Beim DDD wird die Geschäftslogik in eine eigene Schicht, die „Domain Layer“, gelegt. Dies sorgt für eine enge Verbindung zum Modell, wie es das MDD vorsieht. Mit Mustern wie dem Model-View-Controller kann man dann diese Schicht nutzen [Eva07, S. 69 - 73].

Assoziationen im Modell

Wie Assoziationen zwischen Modell und Umsetzung dargestellt werden, kann sich unterscheiden. Man kann diese vereinfachen, indem man z.B. eine bestimmte Richtung festlegt, was das Design klärt und Abhängigkeiten verringert. Ein weiterer Weg ist das Hinzufügen eines Qualifikators, um die Anzahl der Beziehungen zu reduzieren. Zudem kann man unwichtige Assoziationen auch ganz weglassen [Eva07, S. 81-85].

Entities

„Entities“ sind identifizierbare Objekte eines speziellen Bereichs. Wichtig ist, dass man sie einzeln identifizieren kann, nicht ihre spezifischen Merkmale. Dies kann z.B. mit automatisch erzeugten Identifikationswerten geschehen. Sie symbolisieren die durchgehende Existenz eines Objekts über dessen Lebenszyklus. Beim Erstellen eines Modells geht es hauptsächlich darum, diese

Kontinuität zu gewährleisten. Klassendetails, Aufgaben und Assoziationen sollten die Identität der Entity zeigen. Entitäten sollen sparsam verwendet werden. Nur notwendiges Verhalten soll ihnen hinzu gefügt werden. Zusätzliches Verhalten soll zu anderen zugehörigen Objekten verschoben werden [Eva07, S. 89-96] [Das17, S.75].

Value Objects

„Value Objects“ sind Typen ohne eigene Identität, die spezifische Werte repräsentieren. Sie können aus Value Objects Objekten bestehen und Entitäten referenzieren. Oft werden sie als Parameter zwischen Objekten oder als Eigenschaften von Entitäten genutzt. Value Objects sollten ein stimmiges Ganzes bilden und sind idealerweise unveränderlich, wodurch sie sicher geteilt werden können. Bidirektionale Verbindungen bei Value Objects sind aufgrund ihrer fehlenden Identität nicht sinnvoll und sollten vermieden werden [Eva07, S. 97-103] [Das17, S.75].

Services

„Services“ bieten eine spezielle Funktion im Modell an, ohne dabei Daten oder Zustände zu speichern. Er wird nach dem benannt, was er macht, und nicht nach dem, was er ist. Services starten oft bestimmte Vorgänge und arbeiten mit den Objekten des Modells. Sie kombinieren Schritte in Geschäftsprozessen. Sie sollten bedacht eingesetzt werden und dabei nicht alle Aufgaben von Entities und Value Objects abnehmen.

Ein guter Service sollte folgende Eigenschaften erfüllen:

- Er bezieht sich auf ein Konzept, das nicht direkt zu einer Entität oder einem Wertobjekt gehört.
- Es existiert eine klare Schnittstelle im Bezug auf andere Elemente des Modells.
- Kein Zustand wird gespeichert

Da Services auch außerhalb der Domain Layer auftreten können ist eine präzise fachliche Betrachtung nötig [Eva07, S. 104 - 108] [Das17, S.74].

Packages

„Packages“ sorgen im Modell für starke Kohäsion innerhalb und schwache Kopplung zwischen ihnen, indem sie zusammengehörige Elemente bündeln [Eva07, S. 109 - 116].

Aggregates

„Aggregates“ sind Ansammlungen von Objekten, die zusammen als Einheit betrachtet werden. Ihr Zweck ist es, Veränderungen als Ganzes zu handhaben, um Konsistenz zu gewährleisten. Ein direkter Zugriff auf einzelne Bestandteile kann Probleme verursachen. Deshalb gibt es in einem Aggregat ein Hauptobjekt (Wurzel), das für alle Aktionen verwendet wird.

In jedem Aggregat gibt es diese Hauptobjekt und einen definierten Bereich, in dem sich alle zugehörigen Objekte befinden. Andere Objekte können nur auf dieses Hauptobjekt verweisen, wobei die enthaltenen Objekte untereinander verweisen können. Objekte innerhalb des Aggregats, die nicht die Hauptfunktion ausüben, sind nur in ihrem Kontext eindeutig identifizierbar [Eva07, S. 123 - 135] [Das17, S.76].

Factories

Eine Factory erstellt Objekte. Dies ist nützlich, wenn das Erstellen eines Objekts mehr ist als nur das Ausführen eines Konstruktors, da manchmal besondere Regeln oder Logik benötigt werden. Zwei grundlegende Anforderungen an einer Factory sind:

- Sie stellt sicher, dass jedes erstellte Objekt korrekt und vollständig ist.
- Sie fokussiert sich auf den Typ des Objekts, das erstellt werden soll, und nicht unbedingt darauf, wie es gemacht wird.

Auch wenn eine Factory dafür sorgt, dass ein Objekt richtig erstellt wird, kann sie einige Überprüfungen an andere Teile auslagern. Es gibt auch Factories, die bestehende Objekte wiederherstellen. Dabei gibt es Unterschiede, z.B. weisen sie keine neuen IDs zu und sie müssen Verletzungen von Regeln anders behandeln [Eva07, S. 136 - 146] [Das17, S.77].

Repositories

„Repositories“ sind dafür zuständig, Daten von bestimmten Objekten, wie Entitäten oder Aggregaten, sicher zu speichern und zu verwalten. Die Hauptidee von Repositories ist, einen zentralen Ort zu haben, der dafür sorgt, dass alles korrekt und konsequent gespeichert wird. Man kann sich ein Repository wie eine besondere Sammlung vorstellen, aus der man Daten anhand von bestimmten Kriterien abfragen kann. Dabei fügt das Repository neue Objekte der Datenbank hinzu oder löscht sie. Wenn man Daten aus einem Repository möchte, nutzt man spezielle Suchmethoden. Dabei kann die Suche einfach oder auch sehr detailliert sein [Eva07, S. 147 - 161] [Das17, S.76].

2.1.3 Strategic Design

Als „Strategic Design“ wird im DDD beschrieben, wie verschiedene Domänenmodelle miteinander interagieren können. Dabei müssen die strategischen Designprinzipien die Designentscheidungen leiten, um die Abhängigkeiten der Teile zu verringern und die Klarheit zu erhöhen, ohne dabei die kritische Interoperabilität und Synergie zu verlieren [Wol16, S. 45] [Eva07, S. 334].

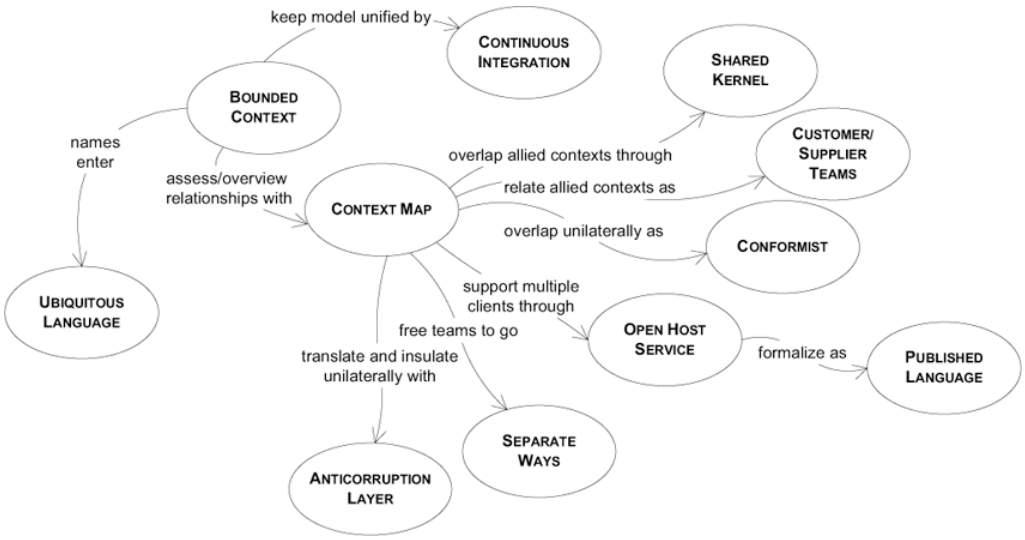

Abbildung 2.2: Supple Design

Bounded Context

Im Zentrum dieser Designentscheidungen steht der „Bounded Context“. Der Grundgedanke dahinter ist, dass jedes Domänenmodell nur in einem klar definierten Rahmen sinnvoll ist [Wol16, S. 45]. Dieser Rahmen könnte ein spezifischer Codebereich oder die Arbeit eines Teams sein. Der Bounded Context beschreibt die Bedingungen, unter denen die Begriffe im Modell eine bestimmte Bedeutung haben. Eine Domäne kann aus mehreren solchen Kontexten bestehen, die auch Informationen mit anderen Kontexten teilen können. Jeder Bounded Context hat eine Schnittstelle, die bestimmt, welche Modelle für andere Kontexte verfügbar sind [New15, S.57].

Ein Bounded Context legt fest, in welchem Bereich ein Modell und somit auch die dazugehörige Ubiquitous Language gültig ist. Das hilft dabei, Verknüpfungen und mögliche Missverständnisse an Schnittstellen zu erkennen. Ein solcher Kontext klärt das Verständnis eines Modells innerhalb eines Teams und wie es sich zu anderen Kontexten verhält. Im Rahmen dieses Kontexts sollte das Modell konsistent gehalten werden, ohne dass man sich Sorgen um die Relevanz außerhalb dieser Grenzen machen muss [Eva07, S. 335 - 340] [Dow18, S.64-65].

Context Map

Eine „Context Map“ liegt an der Schnittstelle zwischen Projektmanagement und Softwareentwurf. Sie ist eine Übersicht über bestehende Modelle, wobei der Fokus auf den Berührungspunkten liegt. Diese Karte macht explizit die Kommunikationswege und gemeinsam genutzten Bereiche deutlich. Context Maps werden verwendet, um die Beziehungen verschiedener Bounded Contexts darzustellen [Eva07, S. 344 - 353] [Wol16, S. 45].

Shared Kernel

Ein Bereich des Domänenmodells kann dazu dienen, zwischen Teams geteilt zu werden. Das betrifft sowohl die Codebasis als auch die Datenbank, die mit dem Modell verknüpft sein könnte. Bei einem solchen „Shared Kernel“ teilen sich die Domänenmodelle einige Elemente, können sich jedoch in anderen Aspekten unterscheiden. Dies kann entweder implizit geschehen oder explizit durch den gemeinsamen Codegebrauch bzw. durch den Zugriff auf dieselben Tabellen in der Datenbank. Oft bezeichnet der Shared Kernel die Kern-Domäne, eine Sammlung generischer Teilbereiche oder beides. Ziel dabei ist es, Wiederholungen zu vermeiden [Eva07, S. 354 - 355] [Wol16, S. 45] [Dow18, S.65].

Customer/Supplier

Zwischen Parteien mit zwei verschiedenen Kontexten kann eine Beziehung entstehen, die aus einer nachgelagerten und einer vorgelagerten Komponente besteht. Das Domain-Driven Design definiert deshalb eine klare Kunden-/Lieferantenbeziehung für solche Kontexte. Die „Customer/Supplier“-Beziehung zeigt, dass ein Untersystem dem Aufrufer ein Domänenmodell zur Verfügung stellt. Der Aufrufer, also der Kunde, legt die Struktur des Modells fest. Der Anbieter der Schnittstelle richtet sich beim Design nach den Wünschen des Kunden und gibt ihm oft ein Mitspracherecht. Es sind automatisierte Tests und Überwachungen notwendig, um Schäden im nachgelagerten Bereich zu vermeiden [Eva07, S. 356 - 360] [Wol16, S. 45,47] [Dow18, S.66].

Conformist

Wenn die Anforderungen eines nachgelagerten, abhängigen Subsystems durch den vorgelagerten Teil gedeckt werden können, bietet es sich an, dem nachgelagerten Teil die Rolle des „Conformist“ zuzuweisen. Dies vereinfacht den Datenaustausch zwischen den beiden Kontexten, indem beide dasselbe Modell verwenden. Der Aufrufer nutzt also das gleiche Modell wie das aufgerufene System und passt sich somit dem anderen Modell an [Eva07, S. 361 - 363] [Wol16, S. 47] [Dow18, S.66].

Anticorruption Layer

Wenn man Systeme kombiniert, die auf verschiedenen Modellen basieren, kann es passieren, dass das Modell des einen Systems durch das andere beeinflusst wird. Ein „Anticorruption Layer“ kann helfen, indem er eine Schutzschicht bildet. Diese Schicht ermöglicht es den Nutzern, mit ihrem gewohnten Modell zu arbeiten, ohne das andere System zu beeinflussen. Sie dient als Übersetzer zwischen den beiden Modellen und sorgt dafür, dass sie unabhängig voneinander bleiben. Die Schicht kommuniziert über eine Schnittstelle mit dem anderen System, ohne dass große Änderungen daran notwendig sind. Sie kann bei Bedarf in beide Richtungen zwischen den Modellen übersetzen [Eva07, S. 364 - 370] [Wol16, S. 47] [Dow18, S.66].

Separate Ways

Wenn der Nutzen im Vergleich zu den Integrationskosten zu gering ist, kann es sinnvoll sein, festzulegen, dass ein bestimmter Kontext keine Verbindung zu anderen hat. Im DDD spricht man hier von „Separate Ways“, was bedeutet, dass die beiden Systeme nicht verbunden werden und unabhängig voneinander bleiben. Anstatt Lösungen für Probleme gemeinsam zu nutzen, müssen diese auf einem klaren, speziellen Weg innerhalb dieses Rahmens umgesetzt werden. Dies kann zum Beispiel mit dem Composite UI Pattern erreicht werden [Eva07, S. 371 - 373] [Wol16, S. 47] [Dow18, S.66].

Open Host Service

Wenn ein Teilsystem mit vielen anderen verbunden werden muss, kann es problematisch sein, für jedes einen Übersetzer anzupassen. Hier bietet es sich an, ein offenes Protokoll von Diensten zu definieren, das Zugang zum Teilsystem ermöglicht. Dieser Kontext, als „Open Host Service“ bezeichnet, stellt also Schnittstellen bereit, die jeder nutzen kann. So kann jeder Bereich seine eigene Anbindung erstellen. [Eva07, S. 374] [Wol16, S. 48] [Dow18, S.66].

Published Language

Das direkte Übersetzen von Modellsprachen kann zu sehr komplexen Strukturen führen. Selbst wenn eine Sprache für den Datenaustausch genutzt wird, ist sie einmal festgelegt und lässt sich nicht leicht an Entwicklungsbedürfnisse anpassen. Deshalb kann eine gut dokumentierte, gemeinsame Sprache als allgemeines Kommunikationsmittel bereitgestellt werden. Diese „Published Language“stellt eine Modellierung der Domäne als universelle Sprache zwischen den Bounded Contexts dar [Eva07, S. 375-378] [Wol16, S. 48] [Dow18, s.65].

2.2 Microservices

2.2.1 Geschichte und Definition

Im Mai 2011 wurde der Begriff „Microservice“ während eines Architekten-Workshops in Venedig eingeführt, um einen neu entstehenden Architekturstil zu beschreiben. Dieser Stil entwickelte sich aus der „Service-orientierten Architektur“ (SOA). Ein Software-Service bezeichnet in der SOA eine Softwarekomponente, die über das Internet von entfernten Computern aus zugänglich ist. Microservice-Architekturen ähneln der SOA, aber SOA wird oft unterschiedlich verstanden und unterscheidet sich in der Regel von Microservices, insbesondere in Bezug auf die Betonung von ESBs. Viele SOA-Projekte stießen auf Schwierigkeiten, sei es wegen ihrer Komplexität oder hohen Kosten. Aufgrund von Herausforderungen bei Skalierung und Verschiebung von Diensten, die nach diesen Prinzipien arbeiteten, entstand ein neuer Ansatz: Ein völlig unabhängiger Dienst mit eigener Datenbank, eigener Benutzeroberfläche und der Fähigkeit, einen Dienst zu ersetzen oder zu replizieren, ohne andere Systemdienste ändern zu müssen.

POINTS OF DIFFERENCE	SOA	Microservices
DESIGN	In SOA, software components are exposed to the outside world for usage in the form of services.	Microservices is a part of SOA. It can be considered as an implementation of SOA.
DEPENDENCY	The business units are dependent.	Microservices do not share a data layer, each microservice is autonomous.
REMOTE SERVICES	SOA relies on multiple message protocols.	Microservices use lightweight protocols & are homogeneous.
SOFTWARE SIZE	The size of the software is larger than that of any conventional software.	Software size is always small in Microservices architecture.
COMPONENT DECOUPLING	One of the significant capabilities offered within SOA.	Microservices on the other hand, do not support contract decoupling.
COMMUNICATION	Uses Enterprise Service Bus (ESB) for communication.	Microservices use simple messaging system.

Abbildung 2.3: Gegenüberstellung von SOA und Microservices

Ein Jahr später wurde der Name Microservices als am besten passend für dieses Konzept gewählt. Der Microservice-Architekturstil wurde dann von Martin Fowler und James Lewis im März 2014 in einem Blogbeitrag erstmals ausführlicher vorgestellt [LF14] [RF21, S.251]. [Som20, S.150,151].

Microservices wurden nicht im Voraus geplant, sondern entstanden durch beobachtete Trends und Muster in der Praxis. Dazu gehören Continuous Delivery, bedarfsorientierte Virtualisierung, automatisierte Infrastrukturen, skalierbare Systeme und unabhängige Entwicklerteams. Die Hauptinspiration stammt jedoch aus dem DDD, insbesondere dem Konzept des Bounded Context [New15, S.21] [RF21, S.251].

Microservices können als kleine, leicht ersetzbare, unabhängige Komponenten definiert werden, die sich auf eng begrenzte Geschäftsvorgänge konzentrieren. Ihr Hauptziel ist eine hohe Entkopplung.[RF21, S.252] [Fow17, S.5,6] [Som20, S.152] [Erl17, S.113] [New15, S.22-23].

2.2.2 Eigenschaften von Microservices

Verschiede Eigenschaften zeichnen einen Microservice aus:

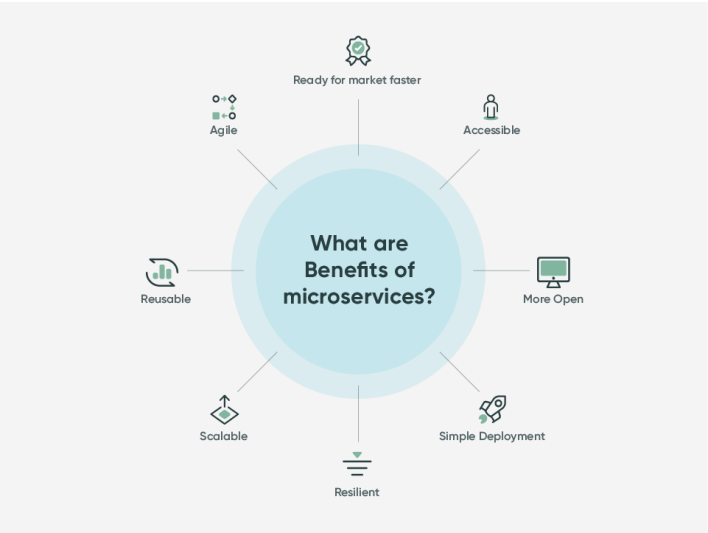

Abbildung 2.4: Eigenschaften von Microservices

Technologisch Uneingeschränkt - In ihrer Implementierung können Microservices unabhängig voneinander mit verschiedenen Technologien umgesetzt werden. Es existieren keine technologische Einschränkungen [Wol16, S. 2, 31-37] [Som20, S.154-157] [New15, S.24].

Unabhängigkeit - Die Dienste können weiterhin unabhängig voneinander überarbeitet und deployt werden, ohne Beeinflussung des gesamten Systems. Build und Deployment sind dabei automatisiert. [Wol16, S. 2, 31-37] [New15, S.28].

Isolierte Daten - Microservices, die auf dem Konzept des Bounded Contexts basieren, legen Wert auf Datenisolierung. Sie vermeiden jegliche Formen der Kopplung, einschließlich gemeinsam genutzter Schemata und Datenbanken als Integrationspunkte, und verfügen über einen eigenen Datenhaushalt auf ihrer ausführenden Infrastruktureinheit [Wol16, S. 2, 31-37] [RF21, S.255].

Eigene Prozesse - Ein Microservice ist wie ein eigenständiges Programm, das entweder als eigener Dienst auf einer Plattform oder als eigener Prozess auf einem Betriebssystem läuft. Jeder dieser Dienste arbeitet unabhängig und kann automatisch und einzeln bereitgestellt werden.[New15, S.23] [RF21, S.253] [Wol16, S. 2, 31-37] [Som20, S.154-157].

Verteilte Netwerkkommunikation - Microservices repräsentieren eine verteilte Architektur und nutzen verteilte Netzwerkkommunikation für die Interaktion mit anderen Diensten. Die Integration der Services erfolgt hauptsächlich über REST oder Lightweight Messaging. Durch die Kommunikation über leichte Protokolle wird der Overhead minimiert. Die Netzwerkkommunikation betont die Isolierung der Services und vermeidet die Gefahren einer engen Kopplung [RF21, S.253] [Wol16, S. 2, 31-37] [Som20, S.154-157] [Dow18, S.128] [New15, S.23].

Einschränkungen bei Transaktionen - Aufgrund dieser Architektur, sollten Transaktionen nicht

über mehrere Services hinweg durchgeführt werden. Transaktionen sind durch die ACID-Eigenschaften (Atomizität, Konsistenz, Isolation, Dauerhaftigkeit) gekennzeichnet [RF21, S.253] [Wol16, S. 2, 31-37].

API-Schicht - In vielen Darstellungen von Microservices ist eine API-Schicht erkennbar, die zwischen den Nutzern (Consumern) und dem System liegt. Obwohl diese Schicht optional ist, wird sie häufig verwendet, da sie in der Architektur gut positioniert ist, um nützliche Aufgaben zu erfüllen. Sie sollte jedoch nicht als Vermittler oder Orchestrierungswerkzeug fungieren, da Microservices streng domänenspezifisch sind. Jeder Service stellt eine API bereit, über die andere Services mit ihm kommunizieren können. Es ist wichtig, dass durch die API keine Kopplung an die Consumer entsteht, was bei der Technologieauswahl zu berücksichtigen ist [RF21, S.255,256] [New15, S.24].

Unabhängigkeit - Microservices sind durch ihre Selbstständigkeit gekennzeichnet und haben keine externen Abhängigkeiten. Sie sind tendenziell kleiner und leicht austauschbar, wobei der Fokus auf Ersetzbarkeit statt auf Wiederverwendung liegt. Der Ansatz von Microservices zielt darauf ab, Services für eng begrenzte Geschäftsvorgänge zu erstellen, die unabhängig voneinander geändert und bereitgestellt werden können, ohne die Consumer zu beeinflussen [Som20, S.154-157] [Dow18, S.128] [New15, S.22,23].

2.2.3 Architektur von Microservice-Systemen

Microservices sind kleine Dienste, die zusammengesetzt Anwendungen bilden. Sie unterscheiden sich von traditionellen, geschichteten Architekturen. Statt einer festgelegten Menge von Komponenten, setzen sie auf flexiblere, unabhängige Dienste. In der Softwareentwicklung gibt es den Trend, Systeme modular zu gestalten. Microservice-Architekturen tun dies, indem sie Software in Dienste statt in klassische Bibliotheken gliedern. Vorteile sind unter anderem die unabhängige Bereitstellung und klare Schnittstellen, die eine enge Verknüpfung vermindern [Som20, S.154-157] [LF14].

DDD-Prinzipien

Der Kerngedanke hinter Microservices ist der Bounded Context. Jeder Dienst repräsentiert eine Domäne oder einen bestimmten Arbeitsprozess. Ein Microservice sollte eine eigenständige fachliche Einheit darstellen, die so gestaltet ist, dass bei Änderungen oder neuen Funktionen nur dieser eine Microservice angepasst werden muss. Ein Bounded Context kann in mehrere Microservices unterteilt werden, falls das zweckmäßig ist. Zudem sollten die aus dem DDD bekannten Konzepte des Strategic Design berücksichtigt werden. Die Zuständigkeiten der Anwendungen müssen daher klar definiert und von anderen Anwendungen abgegrenzt sein. Ebenso sollten die Belange verschiedener Anwendungen auch auf Code-Ebene separat gehalten werden [RF21, S.253,254] [Wol16, S. 48-51] [Das17, S.289].

Kopplung und Kohäsion

Kohäsion beschreibt, wie eng Teile eines Moduls zusammengehören sollten. Es wird gemessen, wie verwandte Teile sich zueinander verhalten. Am besten sind alle Teile eines kohäsiven Moduls im gleichen Package. Teilt man so ein Modul auf, müssen die Einheiten durch Aufrufe zwischen den Modulen verbunden werden, um sinnvolle Ergebnisse zu erzielen [RF21, S.40].

Die Kopplung zeigt, wie stark eine Komponente von einer anderen abhängt. Man unterscheidet zwischen afferenter Kopplung, die eingehende Verbindungen eines Code-Teils zählt, und efferenter Kopplung, die Verbindungen zu anderen Code-Teilen misst [RF21, S.44].

Das Ziel bei Microservices ist, einen Dienst mit starker Kohäsion und schwacher Kopplung zu schaffen. Dies basiert auf dem „Single Responsibility Principle“ [Som20, S.154-157]. Innerhalb eines Dienstes sollten die Bestandteile eng zusammenarbeiten (hohe Kohäsion). Zwischen den Microservices sollte eine schwache Kopplung herrschen, sodass sie unabhängig und leicht änderbar sind. Wenn Dienste schwach gekoppelt sind, bedeutet das, dass eine Änderung in einem Dienst nicht automatisch Änderungen in anderen erfordert. Ein schwach gekoppelter Dienst kennt nur das Nötigste über andere Dienste. Deshalb ist es ratsam, die Kommunikation zwischen den Diensten zu beschränken, um Geschwindigkeitsprobleme und enge Kopplungen zu vermeiden. Zyklische Abhängigkeiten in der Architektur sind problematisch, da sie die

Unabhängigkeit der Änderungen beeinträchtigen [Wol16, S. 101-105] [New15, S.56].

Generell sollten Microservices und ihre Schnittstellen auf spezifische Kontexte ausgerichtet sein. Dies schafft optimale Voraussetzungen für schwache Kopplung und starke Kohäsion [New15, S.59,60].

Ereignisgetriebene Architektur in Microservices

Um gemeinsame Logik zu implementieren, müssen Microservices miteinander kommunizieren. Dadurch werden größere Transaktionen in kleinere aufgeteilt, die zwar für sich genommen inkonsistent sein können, aber im Gesamtkontext für Konsistenz sorgen. Eine ereignisgetriebene Architektur (Event-Driven Architecture, EDA) kann hierbei helfen. Bei dieser Architektur sendet ein Dienst, bei einem bestimmten Ereignis, eine Nachricht aus. Dieser Dienst, der Event-Emitter, informiert so alle beteiligten Microservices darüber, dass eine Aktion stattgefunden hat. Der eventbasierte Architekturstil arbeitet verteilt und asynchron. In dieser Architektur erfolgt die Kommunikation über asynchrone Nachrichten, die zuverlässig gesendet und empfangen werden. Die Komponenten sind entkoppelt und verarbeiten diese Nachrichten asynchron [Wol16, S. 137 - 138] [RF21, S.183] [Das17, S.301].

Für die Ereignisgetriebene Architektur existieren zwei grundlegende Topologien: Die Broker-Topologie und die Mediator-Topologie.

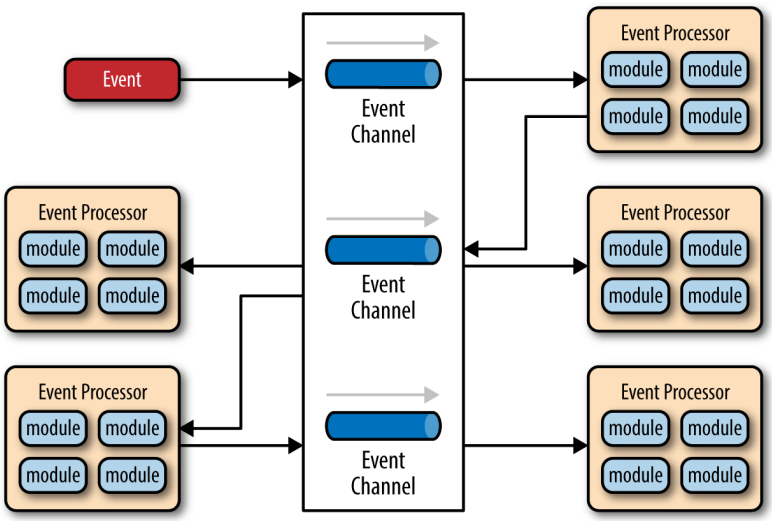

Abbildung 2.5: Broker-Topologie

In der Broker-Topologie verteilt ein einfacher Message Broker Nachrichten über die Event-Prozessorkomponenten, ähnlich einem verketteten Broadcasting. Diese Topologie ist sinnvoll, wenn hohe Reaktionsfähigkeit und dynamische Kontrolle über die Eventverarbeitung gefordert sind. Sie ist besonders nützlich, wenn der Event-Verarbeitungsfluss einfach ist und keine zentrale Orchestrierung oder Koordination benötigt.

Die Broker-Topologie besteht aus vier Hauptkomponenten: Einem auslösenden Event, dem Event-Broker, einem Event-Prozessor und einem verarbeitenden Event.

Das auslösende Event startet den Event-Fluss und wird zur Verarbeitung an einen Kanal des Event-Brokers weitergeleitet. Ein einzelner Event-Prozessor übernimmt und verarbeitet das Event. Nach der Verarbeitung informiert er das System über ein sogenanntes Verarbeitungs-Event über seine Aktionen. Dieses wird asynchron an den Event Broker gesendet, um eventuell weitere Verarbeitungsschritte zu initiieren. Andere Event-Prozessoren reagieren darauf, führen Aktionen aus und informieren das System erneut über ein Verarbeitungs-Event. Dieser Prozess wiederholt sich, bis kein weiteres Interesse an den Aktionen des letzten Event-Prozessors besteht.

Ein Event-Broker ist oft in Domänen unterteilt, wobei jeder Broker alle für eine Domäne notwendigen Kanäle enthält. In der Broker-Topologie wird ein entkoppeltes, asynchrones „fire and forget“ Broadcasting mit Themen verwendet, die nach dem Publish-Subscribe-Modell funktionieren. Jeder Event-Prozessor teilt dabei dem System mit, was er getan hat, unabhängig davon, ob dies für andere Event-Prozessoren relevant ist oder nicht [RF21, S.184-186].

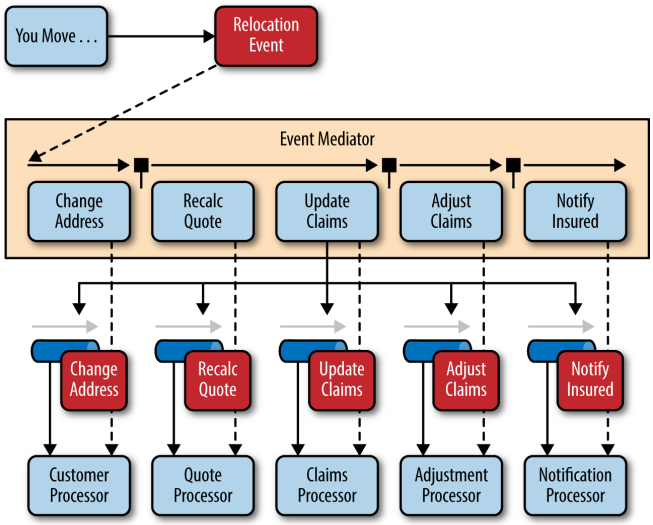

Abbildung 2.6: Mediator-Topologie

Die Mediator-Topologie verfügt über einen Event-Mediator, der den Arbeitsablauf und die beteiligten Event-Prozessoren für initiale Events steuert. Sie setzt sich zusammen aus einem initialen Event, einer Event-Queue, dem Event-Mediator, Event-Channels und Event-Prozessoren.

In dieser Topologie wird das initiale Event zuerst an die Event-Queue gesendet, die vom Event-Mediator verwaltet wird. Der Event-Mediator kennt nur die für die Event-Verarbeitung erforderlichen Schritte und erzeugt daher Verarbeitungs-Events, die zu den jeweiligen Event-Channels weitergeleitet werden. Die Event-Prozessoren hören auf bestimmten Event-Channels, bearbeiten das Event und informieren in der Regel den Event-Mediator nach Abschluss ihrer Aufgabe. Im Unterschied zur Broker-Topologie teilen die Event-Prozessoren in der Mediator-Topologie nicht mit, was sie verarbeitet haben. Häufig gibt es verschiedene Mediatoren, die

jeweils für bestimmte Bereiche oder Event-Gruppen verantwortlich sind. [RF21, S.189-190]

Orchestrierung und Choreographie

Wenn komplexere Programmlogiken entwickelt werden, müssen Geschäftsprozessen abgebildet werden, die über mehrere Services verteilt sind. Meistens handelt es sich um eine Abfolge von Interaktionen, bei denen Operationen in einer bestimmten Reihenfolge stattfinden. Dies nennt man auch Workflow.

Es gibt zwei von der EDA beeinflusste Architekturstile die diese Aufgabe angehen: Orchestrierung und Choreographie.

Bei der Orchestrierung gibt es eine zentrale Stelle, die den Prozess steuert. Ein Workflow kann klar definiert werden, und es gibt einen separaten Dienst, der den Workflow ausführt, indem er die einzelnen Dienste nacheinander aufruft. Durch diesen Mediator entsteht eine Verknüpfung zwischen den Diensten. Gleichzeitig kann der Architekt die Koordination auf einen Dienst konzentrieren, ohne andere zu beeinflussen.

Choreographie hingegen verwendet den Kommunikationsstil der Broker-Topologie. Entsprechend der Philosophie des Bounded Context gibt es keine zentrale Steuerung. Bei der Choreographie wird den Systemkomponenten ihre Aufgabe mitgeteilt und überlassen ihnen die Einzelheiten der Umsetzung. Jeder Dienst teilt ein Ereignis, wenn er fertig ist. Andere Dienste beobachten diese und reagieren darauf. Zur Umsetzung wird zusätzliche Software wie ein Message-Broker benötigt, der Veröffentlichungs- und Abonnierfunktionen unterstützt [New15, S.72] [RF21, S.261-262,264-265] [Som20, S.168-169].

Anpassungen der Microservice-Gesamtarchitektur

Durch neue Erkenntnisse aus der Domäne, wachsenden Code oder auch dem Wunsch Code wiederzuverwenden, kann es notwendig sein Teile der Microservice-Architektur anzupassen. Verschiedene Methoden existieren dazu.

Code, der in mehreren Diensten verwendet wird, kann in Bibliotheken ausgelagert werden. Diese Bibliotheken erleichtern das Teilen von Funktionalitäten mit anderen Teams oder Services. Man muss allerdings auf die entstehenden Abhängigkeiten achten, denn unkoordinierte Änderungen können zu Fehlern im Microservice-System führen. Zudem geht die Möglichkeit, verschiedene Technologien zu nutzen, verloren [Wol16, S. 1121-115] [New15, S.31].

Gemeinsamer Code kann auch durch das Sidecar-Entwurfsmuster wiederverwendet werden. Die Sidecar-Komponente ist ein Bestandteil eines Dienstes und kümmert sich beispielsweise um technische Betriebsaufgaben wie Logging, Monitoring und Sicherung. Wenn jeder Dienst ein Sidecar hat, kann ein Service Mesh erstellt werden, das eine einheitliche Kontrolle über Funktionen wie Logging und Monitoring in der gesamten Architektur ermöglicht. Die typischen Sidecar-Komponenten bieten dabei eine durchgängige Schnittstelle für den technischen Betrieb

aller Microservices. Das Service Mesh gibt damit Entwicklern umfassenden Zugriff auf die Dienste.[RF21, S.256-258].

Das Microservice-Chassis stellt in diesem Kontext eine weiter Lösung für solche gemeinsamen Belange dar, indem es standardisierte Build-Komponenten und Werkzeuge für diese Aspekte bereitstellt. Dies ermöglicht eine effiziente Service-Implementierung ohne redundante Einrichtungsaufgaben. Zudem vereinfacht das Chassis Aktualisierungen, da zentrale Änderungen an alle Dienste weitergegeben werden können. [Ric]

Man kann den Code eines Dienstes in einen anderen übertragen, um beispielsweiße die Kohäsion zu verbessern. Auch kann man alten Code in einen neuen Dienst verschieben oder Änderungen in der Architektur durch einen neu entwickelten Microservice realisieren. [Wol16, S. 115-120]

Zugriff in einer Microservice-Architektur

Die Service-Erkennung (engl.: Service Discovery) läuft in zwei Schritten ab: Zunächst ermöglicht ein Mechanismus einer Instanz die Registrierung, danach kann der Service gefunden werden. Zu den Technologien, die dies ermöglichen, gehören Systeme mit statischer Namensauflösung wie DNS oder solche mit dynamischer Registrierungsmöglichkeit wie Zookeeper, Consul und Eureka. Service Discovery sorgt dafür, dass Microservices sich gegenseitig erkennen können [New15, S.296-301] [Wol16, S. 143 - 146] [Das17, S.329].

Bei hoher Last skalieren Microservices selbstständig, indem mehrere Instanzen die Arbeit teilen. Ein Load Balancer leitet HTTP-Anfragen an diese Instanzen weiter. Jeder Microservice sollte seinen eigenen Load Balancer haben. Technologien dafür findet man in Cloud-Umgebungen oder mit Tools wie Apache httpd, nginx und HAProxy. Für die Stabilität eines Service sollte der Ausfall einer Instanz das System nicht stören. Bei typischen Microservices wird das mit mehreren Instanzen und einem Lastverteiler erreicht. Lastverteiler teilen Anfragen nach einem Algorithmus auf und können kaputte Instanzen ersetzen. Sie ermöglichen auch die Umwandlung von HTTPS in HTTP, was die Systemeinrichtung einfacher macht [Wol16, S. 147 - 150] [New15, S.274-275].

Weiterhin muss Die Sicherheit des gesamten Systems einheitlich gestaltet werden, sodass jeder Nutzerzugriff einheitlich geregelt ist. Dies beinhaltet die Authentifizierung und Autorisierung. Einzelne Microservices sollten sich nicht um die Authentifizierung kümmern. Dies macht ein zentraler Server. Bei der Autorisierung arbeiten verschiedene Teile zusammen, wobei Microservices bestimmen, welche Funktionen bestimmten Nutzergruppen zur Verfügung stehen. Ein Beispiel für die Umsetzung ist das OAuth2-Protokoll. Nach erfolgreicher Authentifizierung und Autorisierung bekommt der Nutzer ein Access Token, das den Zugriff auf Services erlaubt. Dieses Token kann zwischen Microservices ausgetauscht werden, um sichere Anfragen zu ermöglichen und die Sicherheit im gesamten System sicherzustellen [Wol16, S. 153-161].

2.2.4 Kommunikation

Grundsätzlich müssen sich Architekten zwischen synchroner und asynchroner Kommunikation entscheiden. Bei der synchronen Kommunikation muss der Aufrufer eine Antwort vom Aufgerufenen warten. In Microservice-Architekturen kommt dagegen meist eine protokollbewusste heterogene Interoperabilität zum Einsatz. Um eine Kopplung beim technischen Betrieb zu vermeiden, gibt es bei Microservices keinen zentralen integrations-Hub. Jeder Dienst muss selbst wissen, wie er die anderen Dienste aufrufen kann. Daher müssen alle Dienste wissen (oder per Service Discovery herausfinden), welche Protokolle für den Aufruf anderer Dienste verwendet werden müssen. Da Microservices eine verteilte Architektur sind, kann jeder Dienst in einem anderen Techonologiestack geschrieben sein. Interoperabilität beschreibt Dienste die einander aufrufen. Für die asynchrone Kommunikation nutzen Architekten oft Events und Messages. Das heißt intern kommt eine eventbasierte Architektur zum Einsatz bei der in Microservices Choreografie und Orchestrierung die Funktion der Broker- und Mediator-Muster übernehmen [RF21, S.260,261].

Schlüsselentscheidung bei der Wahl eines Standards für die Kommunikation von Microservices: Sollte die Service-Interaktion synchron oder asynchron sein? Sollten Services direkt kommunizieren oder über einen Message-Broker-Middleware? Welches Protokoll sollte für Nachrichten zwischen den Services verwendet werden? Synchroner Austausch ist weniger komplex als asynchroner. Andererseits ist asynchrone Interaktion oft effizienter, da die Services nicht untätig sind, während sie warten. Asynchrone Services sind locker gekoppelt, daher sollte das Ändern dieser Services einfacher sein. [Som20, S.161-163]

Bei der synchronen Kommunikation wird ein Aufruf an einen entfernten Server übermittelt und es wird gewartet, bis die Transaktion abgeschlossen ist. Bei der asynchronen Kommunikation hingegen wartet der Aufrufer nicht auf den Abschluss der Transaktion, und der Programmablauf wird fortgesetzt. Es ist dem Aufrufer möglicherweise egal, ob die Anfrage überhaupt abgeschlossen wird.

Diese beiden Kommunikationsverfahren führen zu zwei unterschiedlichen Begrifflichkeiten hinsichtlich der Art der Kollaboration: Anfrage/Antwort (Request/Response) und ereignisgesteuert (Event-Based). Im Fall von Anfrage/Antwort stellt der Client eine Anfrage und wartet auf die Antwort. Diese Vorgehensweise lässt sich der synchronen Kommunikation zuordnen, kann aber auch in der asynchronen Kommunikation funktionieren. So könnte ich beispielsweise eine Transaktion veranlassen und einen Callback einrichten, sodass der Server mich informiert, sobald meine Transaktion abgeschlossen ist.

Bei der ereignisgesteuerten Kollaboration wird der Vorgang gewissermaßen auf den Kopf gestellt: Statt eines Clients, der mit einer Anfrage um die Erledigung einer Aufgabe bittet, teilt ein Service einfach mit: ”Dies und jenes ist passiertünd geht davon aus, dass ein anderer Service schon wissen wird, wie darauf zu reagieren ist. [New15, S.70-71]

REST

Mit REST (Representational State Transfer) können Microservices untereinander Code aufrufen. Es gibt eine Vielzahl von Ressourcen, die über URIs identifiziert werden können. Die Ressourcen können über eine feste Menge von Methoden manipuliert werden, welche z. B. durch HTTP vorgegeben sind. Ressourcen können unterschiedlich repräsentiert werden. Beziehungen zwischen Ressourcen können durch Links angegeben werden (HATEOAS). Der Server in einem REST-System sollte zustandslos sein. Eine RESTful-HTTP-Schnittstelle kann durch einen Cache ergänzt werden. RESTful HTTP ist synchron. Daher können Ausfälle auftreten, denen mit Timeouts entgegengewirkt werden kann, um Fehlerfortpflanzung zu vermeiden [Wol16, S. 182 - 183].

Am wichtigsten ist das Konzept der Ressourcen. Man kann sich eine Ressource als ein Objekt vorstellen, von dem der Service Kenntnis hat. Wie eine Ressource nach außen hin dargestellt wird, ist vollständig von ihrer internen Darstellung entkoppelt. REST selbst gibt wenig Auskunft über das zugrundeliegende Protokoll. Am häufigsten wird jedoch HTTP verwendet. HTTP legt genau fest, wie Verben (GET, POST, PUT) auf Ressourcen wirken [New15, S.79-80].

Der REST-Architekturstil basiert auf der Übertragung von Darstellungen digitaler Ressourcen von einem Server an einen Client. Ein RESTful-Service nutzt die im HTTP-Protokoll definierten HTTP-Methoden und ist zustandslos. Jede Ressource sollte über eine adressierbare URI mit einer hierarchischen Struktur verfügen. Ressourcen werden üblicherweise im JSON- oder XML-Format repräsentiert. Die Operationen eines RESTful-Services umfassen das Erstellen (implementiert mit POST), Lesen (implementiert mit GET), Aktualisieren (implementiert mit PUT) und Löschen (implementiert mit DELETE). [Som20, S.173-175].

Messaging

Messaging-Systeme stellen eine weitere Möglichkeit dar, die Kommunikation zwischen Microservices zu ermöglichen. Sie basieren auf dem Versand von Nachrichten, die selbst bei einem Netzwerkausfall übermittelt werden können, da die Messaging-Systeme sie zwischenspeichern. Eine korrekte Verarbeitung kann durch Initiierung einer erneuten Übertragung im Fehlerfall garantiert werden. Diese Art von Messaging-Architekturen ist asynchron und auf hohe Latenzzeiten ausgelegt. Der Aufruf eines anderen Services blockiert die weitere Verarbeitung nicht, sodass ein Microservice ohne Wartezeit auf eine Antwort weiterarbeiten kann. Der Sender kennt den Empfänger nicht, weil die Nachricht auf einer Queue oder einem Topic landet, für die sich potenzielle Empfänger registrieren. Dies entkoppelt Sender und Empfänger.

Messaging bietet eine Lösung für transaktionale Systeme mit Microservices, wobei das Senden und Empfangen von Nachrichten Teil einer Transaktion ist. Tritt bei der Verarbeitung einer Nachricht ein Fehler auf, werden alle gesendeten Nachrichten storniert und jegliche Änderungen rückgängig gemacht. Die Empfänger können ebenfalls transaktional abgesichert sein. Entscheidend dabei ist, dass sowohl das Senden und Empfangen von Nachrichten als auch die

Transaktion auf der Datenbank in einer einzigen Transaktion zusammengefasst werden.

Messaging kann durch verschiedene Technologien wie AMQP, Apache Kafka, JMS oder ATOM-Feeds realisiert werden. [Wol16, S. 183 - 187]

Bei der Frage der verfügbaren Technologien sind im Wesentlichen zwei Aspekte zu beachten: Unser Microservice muss Ereignisse auslösen können, und die Consumer müssen feststellen können, dass diese Ereignisse stattgefunden haben. Message-Broker-Systeme wie RabbitMQ versuchen, beide Aufgaben gleichzeitig zu erledigen. Die Auslöser der Ereignisse greifen auf eine API zu, um dem Message-Broker das Auftreten eines Ereignisses mitzuteilen. Der Message-Broker wiederum informiert die Abonnenten, wenn ein Ereignis eingetreten ist. Der Message-Broker kann sogar den Zustand der Abonnenten überwachen und dadurch nachverfolgen, welche Ereignisse ihnen bereits mitgeteilt wurden. Ein anderer Ansatz ist, für das Eintreten von Ereignissen HTTP zu benutzen. ATOM ist eine REST-konforme Spezifikation, die (unter anderem) das zur Veröffentlichung solcher Feeds erforderliche Vokabular definiert.[New15, S.87-88]

Schnittstellenmanagement und Versionierung in Microservices

Jeder Microservice kann eine oder mehrere Schnittstellen für andere Microservices bereitstellen. Außerdem kann ein Microservice Schnittstellen nach außen haben, die Interaktionen außerhalb des Systemkontexts ermöglichen. Wenn ein Microservice eine Schnittstelle ändert, sollte er auch noch die alte Schnittstelle unterstützen. Weiterhin sollten Schnittstellen getrennt umgesetzt werden. Um Änderungen an einer Schnittstelle deutlich zu machen, kann eine Versionsnummer genutzt werden. Semantic Versioning definiert dabei eine Semantik für Versionsnummern, wobei das Format MAJOR.MINOR.PATCH lautet. Dabei sind Änderungen der Major-Version nicht rückwärtskompatibel, Änderungen der Minor-Version bieten neue Features, und Patch-Änderungen beheben Fehler (Bugfixes). Die Version sollte nicht in einer URL, sondern besser in einem Header bei einem REST-Request angegeben werden. [Wol16, S. 190 - 192]

2.2.5 Betrieb und Continious Delivery

Nach der Entwicklung und dem Start eines Systems muss es auf Servern laufen gelassen werden. Das macht eine Überwachung und regelmäßig Updates notwendig. Früher waren der Betrieb und die Entwicklung eines Systems getrennte Bereiche mit unterschiedlichen Aufgaben für die Systemadministratoren und Entwickler. Heutzutage müssen komplexe Systeme, die aus vielen Microservices bestehen und in der Cloud auf mehreren Containern verteilt sind, einheitlich gemanagt werden. Verteilte Systeme arbeiten mit virtuellen Maschinen und Containern, und Tools wie Kubernetes helfen mit Planung, Service-Entdeckung, Lastverteilung und der Verwaltung von Docker-Containern. Alternativ ermöglichen auch Linux-Paketmanager einfachere Installation und Aktualisierung von Software [Som20, S.179,181] [RF21, S.253].

Um auftretende Fehler zu beheben, gibt es verschiedene Möglichkeiten: Rollbacks machen Updates rückgängig, was aber komplexer werden kann, wenn Datenbanken verändert wurden. Roll Forwards dagegen bringen eine neue, verbesserte Version heraus. Continuous Deployment führt Änderungen sofort im Live-System ein, um die Menge der gleichzeitigen Änderungen klein zu halten. Bei Blue/Green-Deployments gibt es zwei Versionen von je einem Microservice, die gleichzeitig laufen. Das erlaubt uns, Änderungen zu testen und bei Bedarf schnell zwischen der alten und neuen Version zu wechseln [Wol16, S. 263 - 269].

Microservices sind unabhängig und laufen getrennt voneinander, sodass jeder in seinem eigenen Container in der Cloud betrieben werden kann. Das erlaubt es, einzelne Services zu stoppen und neu zu starten, ohne den Rest des Systems zu stören. Sie brauchen ihre eigene isolierte Hardware, die durch Virtualisierung bereitgestellt wird. Dies benötigt eine Infrastruktur, die automatisch viele virtuelle Maschinen erstellen kann. Jeder Microservice muss auch seine eigene Lieferkette für Updates haben, ein gründliches Überwachungssystem, um Probleme schnell zu finden, und eine unabhängige Versionsverwaltung, die verschiedene Versionen zulässt [Som20, S.158] [Wol16, S. 80-81].

Monitoring in der Softwaretechnik bezieht sich auf das Überwachen von Metriken, um den Zustand einer Software zu erfassen und zu verstehen. Zu den Schlüsselmetriken gehören Host- und Infrastrukturmetriken wie CPU- und RAM-Nutzung, Threads, offene Dateideskriptoren und Datenbankverbindungen jedes Microservice. Zudem sind Microservice-spezifische Metriken wie Service-Level-Agreements (SLAs), Latenz, API-Endpunkt-Erfolg und -Antwortzeiten, Quelle und Ziel von API-Anfragen, Fehler sowie der Status von Abhängigkeiten zu berücksichtigen. Ein Monitoring von einem Microservice stütz sich auf vier Hauptkomponenten:

- Die Protokollierung aller relevanten Informationen, die ein Nachvollziehen des Dienstzustandes in Echtzeit sowie retrospektiv ermöglicht.
- Gut gestaltete Dashboards, die den Gesundheitszustand des Microservices präzise anzeigen und für eine klare Übersicht sorgen.
- Eine effektive Alarmierung, die bei Abweichungen von Schlüsselmetriken aktiv wird und Entwicklern hilft, frühzeitig auf potenzielle Probleme zu reagieren.

- Die Etablierung einer Bereitschaftsrotation, die sicherstellt, dass kontinuierlich eine Überwachung und gegebenenfalls schnelle Reaktion gewährleistet ist.

Es ist zudem sinnvoll, eine einheitliche Abfrageschnittstelle für alle Microservices zu implementieren. Da Microservices dynamisch sind und ihre Instanzen starten oder beendet werden können, muss das Monitoring systemübergreifend und dienstunabhängig gestaltet sein. Dies kann durch den Einsatz von Technologien wie Graphite, Grafana oder Nagios realisiert werden, wobei auch Cloud-Umgebungen entsprechende Tools bereitstellen, um ein effizientes Monitoring sicherzustellen [Wol16, S. 250 - 256] [Fow17, S.105-108].

Ein Microservice-System muss jede Anfrage von Anfang bis Ende verfolgen können, um den Datenfluss zwischen verschiedenen Diensten zu verstehen. Dazu nutzt man eine Korrelations-ID, die jede Anfrage identifizierbar macht und über alle Dienste hinweg verfolgt werden kann. Mit Logging können die Dienste dann wichtige Ereignisse dokumentieren. Tools wie Logstash, Elasticsearch und Kibana helfen dabei, Logs aller Microservices zentral zu sammeln und auszuwerten. Während Infrastruktur-Werkzeuge sich um die Überwachung der Infrastruktur kümmern sollten, ist es wichtig, dass Microservices selbst Details wie verschlüsselte Nutzer-IDs und Informationen zu Anfragen und Antworten in ihren eigenen Logs festhalten [Wol16, S. 244 - 249] [Fow17, S.109-110].

2.3 Cloud-Infrastruktur

2.3.1 Definition

Unter Cloud-Computing versteht man das Nutzen von Computing-Ressourcen über das Netzwerk. Durch Virtualisierung können so Ressourcen eines von Cloud-Anbietern gehosteten Rechenzentrums gekapselt werden. In einer Cloud-Umgebung können Konzepte wie Continuous Delivery umgesetzt werden. [Ste20, S. 20]

Softwarecontainer sind eine Methode, um eine beliebige Anwendung in eine normierte, portable Einheit zu verpacken, die ein lauffähiges Gebilde ist. Die zentrale Eigenschaft einer solchen Umgebung ist es, dass der Container als Prozesskapsel von dem restlichen System isoliert ist und im Rahmen bestimmter Grenzen läuft. Dies erlaubt sowohl unterschiedliche Applikationen als auch verschiedene Komponenten derselben Applikation in separaten Containern zu betreiben. Ein Effekt davon ist, dass diese in der Cloud dann unabhängig voneinander skaliert werden können. Diese Modularisierung wird insbesondere bei Microservices als Prinzip für Entwicklung und Applikationsarchitektur auf die Spitze getrieben.

Für die Orchestrierung einer komplexen containerisierten Anwendung wird ein Cluster-Manager benötigt. Cluster bezeichnet dabei eine zusammengehörige Gruppe von Servern mit Containern, während die Instanzen, die in einem solchen Konstrukt eingebunden sind, als Knoten (Nodes) bezeichnet werden. Beispiele dafür sind Swarm Mode (Teil von Docker) oder Kubernetes. [Ste20, S. 46-63]

The NIST (National Institute of Standards and Technology) defines Cloud Computing as follows: ”Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort or service provider interaction. This cloud model is composed of five essential characteristics, three service models (SaaS, PaaS, IaaS), and four deployment models (Private, Community, Public, Hybrid).”[Rit18, S.19-22]

2.3.2 Docker

Ein Container-Image ist ein Binary-Paket, das alle für das Ausführen eines Programms in einem OS-Container notwendigen Dateien zusammenfasst. [BBHE23, S.16]

Das Docker-Image-Format ist der De-facto-Standart. [BBHE23, S.17].

Container gehören zu einer von zwei Hauptkategorien: System-Container oder Anwendungscontainer. System-Container versuchen virtuelle Maschienen nachzubilden, und führen häufig einen vollständigen Boot-Prozess durch. Anwendungs-Container unterscheiden sich von System-Containern darin, dass sie im Allgemeinen nur ein einzelnes Programm in sich laufen lassen [BBHE23, S.18].

Ein Dockerfile kann genutzt werden, um das Erstellen eines Docker-Container-Image zu auto-

matisieren. [BBHE23, S.18]

Containerization can be viewed as an evolution of virtualization. With virtualization, we typically recreate an entire operating system and host it on a host machine. Docker is perhaps the most well-known software for containerization. It provides a level of virtualization known as operating-system-level virtualization, which is synonymous with containerization. This form of isolation enables us to run multiple operating systems within another. [Rit18, S.63]

Docker is composed of the following different components:

The Docker engine is a client-server application. The client talks with the server application, called a daemon. The daemon is responsible for executing the containers. Both the client and the daemon can be on the same machine or on different ones.

Images are the foundation of our Docker architecture. Images are used to launch our containers, and we can create personal images by starting with another basic image.

The registry is where the images are stored. We can identify two types of registries: private and public.

Containers are essentially an image that is executed. A container can have more than one running process inside, depending on how the images are designed and created. Essentially, a container is a lightweight, standalone application. We can combine more than one container to execute a complex application.[Rit18, S.64-65]

2.3.3 Kubernetes

Kobernetes ist ein Open-Source-Orchestrierer für das Deployen containerisierter Anwendungen. Es wurde ursprünglich von Google entwickelt und ist durch ein Jahrzehnt Erfahrung beim Deployen skalierbarer, zuverlässiger Systeme in Containern über anwendungsorientierte APIs inspiriert. [BBHE23, S.1] [BGO+16]

Kubernetes ist ein moderner Standard für den verteilten Betrieb von containerisierten Applikationen auf Infrastrukturen aus der Public Cloud. Kubernetes nutzt Docker als Default-Container-Engine. Kubernetes besitzt eine eigene API, welche vor allem bei der Entwicklung von eigenen Werkzeugen genutzt wird. Das Tool kubectl spricht die Cluster über die auf dem Master laufende API an. Der Master steuert alle angeschlossenen Worker-Nodes und setzt die vom Anwender gewünschten Veränderungen um. [Ste20, S. 166-192]

Alles in Kubernetes ist ein deklaratives Konfigurations-Objekt, das den gewünschten Status des Systems präsentiert. Die Idee, deklarative Konfiguration unter Versionsverwaltung zu stellen, wird oft als Ïnfrastructure as Code"bezeichnet [BBHE23, S.4]

Objekte in Kubernetes sind eine Reihe von verschiedenen Dingen, die der Nutzer im Cluster betreiben kann. [Ste20, S. 166-192]

In Kubernetes wird alles als REST-Ressource repräsentiert. Jedes-Kubernetes Objekt existiert

unter einem eindeutigen HTTP Pfad. [BBHE23, S.40] Objekte können mit Labeln Anmerkungen hinzugefügt werden. [BBHE23, S.42]

Ein Kubernetes-Cluster besteht aus veschiedenen Komponenten: Kubernetes Proxy, Kubernetes-DNS und Kubernetes UI. [BBHE23, S.36-37]

Kubernetes nutzt Namensräume, um Objekte im Cluster zu organisieren. Mit einem Kontext kann der Standart Namenraum geändert werden.[BBHE23, S.39]

Pods sind kleine Organisationseinheiten, deren Aufgabe es ist, Container zu enthalten. Pods sind Objekte die vervielfältigt werden können, um die darin enthlaltene Appliaktion im Cluster zu skalieren. [Ste20, S. 166-192]

Pods sind Gruppen von Containern - können von verschiedenen Teams entwickelte Container-Images zu einer einzelnen deploybaren Einheit verbinden. [BBHE23, S.8]

Ein Pod repräsentiert eine Gruppe von Anwendungs-Containern und Volumes, die in der gleichen Ausführungsumgebung laufen. Pods sind das kleinste deploybare Artefakt in einem Kubrenetes Cluster. Anwendungen die im gleichen Pod laufen, haben die gleiche IP-Addresse und den gleichen Port-Space, den gleichen Hostnamen und sie können über native Interprozess-Kommunikationskanäle kommunizieren. Pods werden in einem Pod-Mainfest beschrieben [BBHE23, S.48,49].

Ein Deployement ist ein Objekt, welches etwas im Cluster installiert um es dort laufen zu lassen. [Ste20, S. 166-192]

Services bieten Load Balancing, Naming und Discovery, um einen Microservice von anderen zu isolieren. [BBHE23, S.8]

2.4 Modellgetriebene Entwicklung von Microservices

Model Driven SOA ist eine Variante der MID-Modellierungsmethodik M³ . Sie unterstützt den modellgetriebenen Entwurf und die Entwicklung von SOA-Anwendungen. Die Methodik wird üblicherweise in mehreren Iterationen durchlaufen. MDSOA läuft in verschiedenen Phasen ab:

Initiationsphase: Aufnahme der Fachlichkeit und deren Anforderungen sowie die Identifikation von fachlichen Services.

Systemevaluierungsphase: Spezifikation der technischen Services, des Fachdatenmodells und der Maskenflüsse.

Architekturprojektionsphase: Abbildung der gewählten Architektur und die Übertragung der in der Initiations- und Evaluierungsphase spezifizierten Inhalte auf konkrete Elemente der Architektur.

Softwarekonstruktion: Generierung von Artefakten und Integration aller Ergebnisse zu einer lauffähigen Anwendung auf einer spezifischen Plattform.

[rem11, S.62-66]

Herausforderungen beim Domain-Driven Microservice Design:

- Ableitung von Microservices aus Domänenmodellen
- Fehlende Infrastrukturkomponenten in Domänenmodellen
- Autonome Domänenmodellierung

Mittel der modellgetriebenen Entwicklung zur Bewältigung von Herausforderungen

- Zwischenmodelle, die Microservice-Schnittstellen und -Bereitstellung spezifizieren.
- Reduzierung der Inoffizialität von Domänenmodellen durch Modellierungskonventionen
- Richtlinien für Autonome Domänenmodellierung

[RSS18]

Tools:

AjiL is an Eclipse-based tool for graphical MSA modeling. It features a diagram editor for modeling rudimentary domain models, functional and infrastructure microservices, and provided and required interfaces. Thus, AjiL facilitates creating intermediate models for service interfaces (see the section “Intermediate Models”). AjiL ships with a code generator that produces microservice implementations from AjiL models. The generated Java code is based on Spring Boot and Spring Cloud. It is directly runnable and comprises interface implementations, stubs for implementing services’ business logic, and configuration files for infrastructure components. However, AjiL currently does not support round-trip engineering.4 The tool is available at github.com/SeelabFhdo/AjiL. [RSS18]

Open Cloud Computing Interface (OCCI) is proposed as a community-based and open recommendation standard for managing any kind of cloud resources. Currently, only runtime

implementations exist for OCCI, and each one targets a specific cloud service model such as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), or Software as a Service (SaaS). Thus, OCCI lacks an approach to model and execute different OCCI artifacts. Our approach provides a generic modeling framework coupled with a generic runtime implementation. In this article, we propose the OCCIware approach, which represents the first approach to design, validate, generate, implement, deploy, execute, and supervise everything as a service with OCCI. This approach provides OCCIware Studio, the first model-driven tool chain for OCCI. It is built around OCCIware Metamodel, which defines the static semantics for the OCCI standard in Ecore and OCL. In addition, it proposes OCCIware Runtime, the first generic OCCI runtime implementation targeting all the cloud service models (IaaS, PaaS, and SaaS). OCCIware provides a unique and unified framework to manage OCCI artifacts and, at the same time, it represents a factory to build cloud domain-specific modeling frameworks where each framework targets a specific cloud domain. OCCIware has been applied in various cloud domains and use-cases, which validate its applicability. [ZCM19]

2.5 Noch zu sichten

2.5.1 ARGON - automated cloud infrastructure provisioning using a domain-specific language

Infrastructure as Code (IaC) is an approach to infrastructure automation based on practices from software development. IaC tools use scripts to specify the creation, update, and execution of cloud infrastructures. As each cloud provider offers a different type of infrastructure, the definition of an infrastructure element implies to write several lines of code that strongly depend on the target cloud provider. As a result, managing IaC scripts has become a time-consuming and error-prone activity. In previous work, we have presented ARGON, which is a model-driven infrastructure provisioning tool. ARGON implements a domain-specific language (DSL) for modeling the characteristics of the cloud infrastructure and provides transformation engines to automate the infrastructure provisioning for different cloud providers. In this demonstration, we show the use of ARGON for modeling and provisioning a load balancer in Amazon Web Services and Microsoft Azure. The load balancer distributes incoming application traffic across multiple virtual machines.

https://ieeexplore.ieee.org/abstract/document/8904494

2.5.2 model-driven approach for adaptive microservice-based systems using VxBPMN4MS

A microservice-based system is composed of numerous independently deployed and executed microservices, among which normally exist the complex dependencies. Traditional service com-

position approaches usually expect the business process predefined at design time. As a result, it is difficult for the microservice-based system to quickly adapt to the frequently changing operation environments and business requirements. To address the above limitations, we propose a variability-enabling and model-driven approach to developing adaptive microservice-based systems. Our approach first models the business process with variability using VxBPMN4MS, an extension of Business Process Model and Notation (BPMN) with support for variability, then transforms the business process model to variability supported microservice composition frameworks, and finally derives business process instances at run-time according to the predefined process configuration. We have developed a platform to automate the proposed approach as much as possible, and conducted a case study to evaluate the effectiveness of the proposed approach and platform.

https://ieeexplore.ieee.org/abstract/document/9529482

2.5.3 Context Mapper, a tool for DDD and microservice design.

Service-oriented architectures and microservices have gained much attention in recent years; companies adopt these concepts and supporting technologies in order to increase agility, scalability, and maintainability of their systems. Decomposing an application into multiple independently deployable, appropriately sized services and then integrating such services is challenging. With strategic patterns such as Bounded Context and Context Map, Domain-driven Design (DDD) can support business analysts, (enterprise) architects, and microservice adopters. However, existing architecture description languages do not support the strategic DDD patterns sufficiently; modeling tools for DDD primarily focus on its tactical patterns. As a consequence, different opinions on how to apply strategic DDD exist, and it is not clear how to combine its patterns. Aiming for a clear and concise interpretation of the patterns and their combinations, this paper distills a meta-model of selected strategic DDD patterns from the literature. It then introduces Context Mapper, an open source project that a) defines a Domain-specific Language (DSL) expressing the strategic DDD patterns and b) provides editing, validation, and transformation tools for this DSL. As a machine-readable description of DDD, the DSL provides a modeling foundation for (micro-)service design and integration. The models can be refactored and transformed within an envisioned tool chain supporting the continuous specification and evolution of Context Maps. Our validation activities (prototyping, action research, and case studies) suggest that the DDD pattern clarification in our meta-model and the Context Mapper tool indeed can benefit the target audience.

https://www.scitepress.org/Papers/2020/89105/89105.pdf

2.5.4 DOML simplifies infrastructure deployments by unifying multiple IaC languages

One of the main DevOps practices is the automation of resource provisioning and deployment of complex software. This automation is enabled by the explicit definition of Infrastructure-as-Code (IaC), i.e., a set of scripts, often written in different modelling languages, which defines the infrastructure and applications to be deployed.

We introduce the DevOps Modelling Language (DOML), a new Cloud modelling language for infrastructure deployments. DOML is a modelling approach that can be mapped into multiple IaC languages, addressing infrastructure provisioning, application deployment and configuration at once. The idea behind DOML is to use a single modelling paradigm which can help to reduce the need of deep technical expertise in using different specialised IaC languages.

We present the DOML's principles and discuss the related work on IaC languages. We demonstrate the DOML advantages for the end-user in comparison with state-of-the-art IaC languages such as Ansible, Terraform, and Cloudify, and show its effectiveness through an example.

Literaturverzeichnis

[BBHE23] Brendan Burns, Joe Beda, Kelsey Hightower, and Lachlan Evenson. *Kubernetes: eine kompakte Einführung*. dpunkt.verlag, Heidelberg, 3., aktualisierte und erweiterte auflage edition, 2023. Brendan Burns, Joe Beda, Kelsey Hightower, Lachlan Evenson.

[BGO+16] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes: Lessons learned from three container-management systems over a decade. *Queue*, 14(1):70–93, jan 2016.

[Das17] Sebastian Daschner. *Architecting modern Java EE applications: designing lightweight, business-oriented enterprise applications in the age of cloud, containers, and Java EE 8*. Packt, Birmingham, 2017. Sebastian Daschner ; Foreword by: David Blevins, Founder and CEO, Tomitribe.

[Dow18] Herbert Dowalil. *Grundlagen des modularen Softwareentwurfs: der Bau langlebiger Mikro- und Makro-Architekturen wie Microservices und SOA 2.0*. Hanser, München, 2018. Herbert Dowalil.

[Erl17] Thomas Erl. *Service-oriented architecture: analysis and design for services and microservices*. The Prentice Hall service technology series from Thomas Erl. Prentice Hall ; Service Tech Press, Boston, second edition edition, 2017. Thomas Erl ; with contributions by Paulo Merson and Roger Stoffers.

[Eva07] Eric Evans. *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley, Upper Saddle River, NJ [u.a.], 11. printing edition, 2007. Eric Evans.

[Fow17] Susan J. Fowler. *Production-ready microservices: building standardized systems across an engineering organization*. O´Reilly, Beijing, first edition edition, 2017. Susan J. Fowler.

[LF14] James Lewis and Martin Fowler. Microservices - a definition of this new architectural term, 2014. Accessed: 01.11.2023.

[New15] Sam Newman. *Microservices: Konzeption und Design*. mitp-Verl., [Frechen], 1. aufl. edition, 2015. Sam Newman. Übers. aus dem Amerikan. von Knut Lorenzen.

[rem11] *Model Driven SOA: anwendungsorientierte Methodik und Vorgehen in der Praxis*. Xpert.press. Springer, Berlin, 2011. Gerhard Rempp; Mark Akermann; Martin Löffler;

Jens Lehmann. Mit Ill. von Martin Starzmann.

[RF21] Mark Richards and Neal Ford. *Handbuch moderner Softwarearchitektur: Architekturstile, Patterns und Best Practices*. O'Reilly, Heidelberg, 1. auflage edition, 2021. Mark Richards und Neal Ford ; deutsche Übersetzung von Jørgen W. Lang.

[Ric] Chris Richardson. Pattern: Microservice chassis. Accessed: 01.11.2023.

[Rit18] Pierluigi Riti. *Pro DevOps with Google Cloud Platform: with Docker, Jenkins, and Kubernetes*. For professionals by professionals. Apress, [Berkeley, California], 2018. Pierluigi Riti.

[RSS18] Florian Rademacher, Jonas Sorgalla, and Sabine Sachweh. Challenges of domain-driven microservice design: A model-driven perspective. *IEEE Software*, 35(3):36–43, 2018.

[Som20] Ian Sommerville. *Engineering software products: an introduction to modern software engineering*. Pearson, Hoboken, NJ, 2020. Ian Sommerville.

[Ste20] Daniel Stender. *Cloud-Infrastrukturen: das Handbuch für DevOps-Teams und Administratoren*. Rheinwerk computing. Rheinwerk Computing, Bonn, 1. auflage edition, 2020. Daniel Stender.

[Wol16] Eberhard Wolff. *Microservices: Grundlagen flexibler Softwarearchitekturen*. dpunkt.verlag, Heidelberg, 1., korrigierter nachdruck edition, 2016. Eberhard Wolff.

[ZCM19] Faiez Zalila, Stéphanie Challita, and Philippe Merle. Model-driven cloud resource management with occiware. *Future Generation Computer Systems*, 99:260–277, 2019.